US Stock Portfolio Analyst Agent — *I used the Sample Prompt Shared in the Project to create the project. I was able to setup Cursor environment, to build the application with advised stack. The change I made was – I used OPENAI model with Calude Code to render the application.*

Executive Overview

The AI-Powered US Stock Portfolio Analyst Agent is a production-grade web application that converts raw transaction history into portfolio metrics, interactive visual analytics, and AI-driven portfolio insights.

Built as a single-page Streamlit application, it processes user-uploaded CSV transaction files, enriches positions with real-time market data, benchmarks performance against the S&P 500, and connects to OpenAI for contextual financial analysis.

Technology Stack & Environment

- **Environment & Package Management:** uv (Fast Python package installer and virtual environment builder)
- **Frontend UI Framework:** Streamlit
- **Market Data Provider:** yfinance
- **AI Engine:** OpenAI API (gpt-4o-mini / gpt-4o)
- **Data Processing & Analytics:** pandas, numpy
- **Data Visualization:** Plotly Express & Plotly Graph Objects
- **Configuration:** python-dotenv for API key security

File Structure & Module Definitions

File Name / Path	Module Role & Definition
pyproject.toml	Project definition file managed by uv. Defines required dependencies, Python version constraints, and virtual environment settings.
.env	Local environment configuration file containing sensitive secrets, specifically the OpenAI API key.

File Name / Path	Module Role & Definition
app.py	Main Streamlit interface entry point. Manages application state, navigation, tab layouts, user inputs, and visual components.
core/data_loader.py	CSV ingestion module. Verifies input file headers, enforces column data types, validates positive values, and sorts transactions chronologically.
core/portfolio_engine.py	Financial logic engine. Calculates active ticker quantities, weighted average cost basis, market value, realized P&L, unrealized P&L, and percentage allocation.
core/market_data.py	Live data fetcher. Interacts with yfinance to retrieve real-time stock prices, previous closing prices, and sector metadata with response caching.
core/ai_agent.py	Conversational AI bridge. Serializes current portfolio metrics into structured JSON context and injects it into OpenAI system prompts to power the AI chat interface.
data/sample_transactions.csv	Test dataset structured with required column headers (ticker, date, transaction_type, quantity, price).

Core Data Processing Pipeline

1. **Ingestion & Validation:** User uploads a CSV; the system validates schema formatting and sorts transactions by date.
2. **Holdings Aggregation:** Transactions are processed sequentially to compute remaining shares and weighted cost basis for each ticker.
3. **Market Enrichment:** The system fetches current stock quotes and sector metadata via yfinance.
4. **Metric Calculation:** Market values, unrealized gains/losses, and percentage allocations are calculated dynamically.
5. **AI Context Injection:** Serialized portfolio state is attached to user chat messages, enabling context-aware conversational responses from OpenAI.